DIPLOMADO EN MACHINE LEARNING Y DATA SCIENCE AVANZADO · DEEP LEARNING

Tarea 1 · Clasificación lineal

Regresión logística con pérdida ponderada sobre el dataset Wine, en Keras



Guía de estudio: el problema, la teoría de fondo, el código explicado y la intervención

Módulo 5, unidad 1 · Universidad Nacional de Colombia, Facultad de Ingeniería

Notebook: `v2_M5U1_Tarea_1_Resuelto.ipynb`

Natalia Patricia Remolina Rodríguez · 7 de noviembre de 2026

✓ Calificación en UNCode: 100 %, 12 de 12 pruebas en 100

La tarea en una página

El problema

- Distinguir dos tipos de vino a partir de **13 variables químicas**.
- Las clases están **desbalanceadas**: 50 frente a 33 muestras en entrenamiento.
- 83 muestras para entrenar y 36 para probar.

La solución

- Una **regresión logística**: una neurona `Dense(1)` con sigmoide.
- Una **entropía cruzada ponderada** que hace pesar más los errores en la clase minoritaria.
- Entrenamiento con Adam, tasa 0,5 y 10 épocas.

La intervención

- Dos **firmas de la plantilla** no coincidían con las pruebas y se corrigieron.
- Cada función debe **importar sus librerías**: el calificador las ejecuta aisladas.
- La pérdida se reescribió con `keras.ops` respetando el **tipo de dato**.

El resultado

- Las 12 pruebas en 100 y envío oficial con **100 %**.
- **98,8 %** de accuracy en entrenamiento y **91,7 %** en prueba.
- Ningún vino de la clase minoritaria se clasificó mal.

CÓMO LEER ESTA GUÍA

La guía avanza de lo general a lo concreto:

- La **parte I** plantea el problema.
- La **parte II** explica la teoría necesaria.
- La **parte III** recorre el código.
- La **parte IV** cuenta qué hubo que corregir.
- La **parte V** interpreta los resultados.
- La **parte VI** cierra con preguntas de autoevaluación.

Las cajas de colores tienen siempre el mismo sentido:

QUÉ SE PEDÍA
la instrucción del enunciado.

INTUICIÓN
la idea detrás del código.

INTERVENCIÓN
lo que hubo que cambiar para llegar al 100 %.

OJO
un detalle fácil de pasar por alto.

Contenido

Parte I · El problema

1. Qué pide la tarea
2. Cómo califica UNCode

Parte II · La teoría de fondo

3. La regresión logística como neurona
4. Entropía cruzada y pesos por clase
5. Cómo se entrena

Parte III · La solución celda por celda

6. Preparación de los datos
7. Punto 1 · class_weights
8. Punto 2 · get_loss
9. Punto 3 · optimizer
10. Punto 4 · get_model
11. Punto 5 · train_model
12. Punto 6 · model_predict

Parte IV · La intervención

13. Las cuatro correcciones
14. Lecciones

Parte V · Resultados

15. Las pruebas
16. Curva de pérdida
17. Errores y variables
18. ¿Sirvió ponderar?

Parte VI · Para consolidar

19. Preguntas de autoevaluación
20. Glosario
21. Conclusiones

PARTE I

El problema

Qué hay que resolver, con qué datos y bajo qué reglas de calificación.

1. Qué pide la tarea

Es un problema de **clasificación binaria**: a partir de 13 mediciones químicas, decidir si un vino es del tipo 2 o del tipo 3 del dataset *Wine* de la UCI. La tarea pide hacerlo con una **regresión logística en Keras** y compensar el desbalance entre clases **desde la función de pérdida**.

Aspecto	Detalle
Datos	178 vinos y 13 variables: alcohol, ácido málico, flavonoides, intensidad de color, prolina y otras. Solo se usan las clases 2 y 3, recodificadas como 0 y 1: 119 muestras
Preparación	Escalado a [0, 1] con `MinMaxScaler` y división estratificada 70 / 30
Particiones	83 de entrenamiento: 50 de la clase 0 y 33 de la clase 1. 36 de prueba: 21 y 15
Modelo	Una neurona con sigmoide: 13 pesos y 1 sesgo, 14 parámetros

Los seis puntos calificados

Punto	Función	Qué debe hacer	Cómo se verifica
1	`class_weights`	Calcular el peso de cada clase según el desbalance	`[0.83, 1.25758]`
2	`get_loss`	Devolver una función de entropía cruzada ponderada	0,6931 con predicciones de 0,5
3	`optimizer`	Devolver SGD, Adam o RMSprop con la tasa indicada	Un objeto Adam
4	`get_model`	Definir y compilar la regresión logística	14 parámetros
5	`train_model`	Entrenar y devolver pérdidas por época y pesos	Pérdidas y pesos exactos
6	`model_predict`	Devolver las probabilidades del modelo	Primeras predicciones y accuracy

2. Cómo califica UNCode

Cada punto tiene dos pruebas. El calificador importa cada función y la ejecuta con sus propias entradas, y compara resultados exactos: los pesos con cinco decimales, el valor de la pérdida, el número de parámetros, las pérdidas por época y las predicciones.

OJO · EL CALIFICADOR EJECUTA CADA FUNCIÓN AISLADA

No ve los `import` que hay arriba en el notebook. Si una función usa `tf` o `keras` sin importarlos dentro, la prueba falla con `name 'tf' is not defined`. Fue una de las correcciones de la parte IV.

PARTE II

La teoría de fondo

Lo mínimo que hay que entender para que cada línea del código tenga sentido.

3. La regresión logística como neurona

Una neurona calcula una suma ponderada de las entradas más un sesgo, `z = w · x + b`, y la pasa por una función de activación. Con la **sigmoide**, el resultado queda entre 0 y 1 y se interpreta como la probabilidad de la clase 1:

```
ŷ = σ(z) = 1 / (1 + e^(−z))
```

z	σ(z)	Lectura
−2	0,12	Probablemente clase 0
0	0,50	Indeciso
2	0,88	Probablemente clase 1

Con 13 variables, la neurona tiene 13 pesos y un sesgo. En Keras es una sola capa `Dense(1, activation='sigmoid')`: **una regresión logística es la red neuronal más pequeña posible**.

INTUICIÓN · QUÉ SIGNIFICA CADA PESO

Como las variables están escaladas a [0, 1], el tamaño de cada peso dice cuánto influye. Un peso positivo empuja hacia la clase 1 y uno negativo, hacia la clase 0. La sección 17 muestra los pesos aprendidos.

4. Entropía cruzada y pesos por clase

4.1 La entropía cruzada binaria

Mide qué tan lejos está la probabilidad predicha de la etiqueta real. Para una muestra:

```
pérdida = −[ y · log ŷ + (1 − y) · log(1 − ŷ) ]
```

Si la etiqueta es 1, solo cuenta `−log ŷ`: vale 0 si ŷ = 1 y crece mucho si ŷ se acerca a 0. Si la etiqueta es 0, cuenta `−log(1 − ŷ)`.

4.2 El problema del desbalance

Con 50 muestras de la clase 0 y 33 de la clase 1, la clase mayoritaria domina la suma de errores. El modelo podría sacrificar la clase minoritaria y aun así reducir la pérdida total. La solución es **multiplicar el error de cada clase por un peso**:

$$w_k = (n_0 + n_1) / (2 \cdot n_k)$$

Clase	Muestras	Cálculo	Peso
0, vino tipo 2	50	83 / 100	0,830
1, vino tipo 3	33	83 / 66	1,258

Equivocarse con la clase 1 cuesta 1,52 veces más que con la clase 0. Además, **el peso promedio por muestra es exactamente 1**: 50 · 0,83 + 33 · 1,258 = 83. Así la pérdida ponderada queda en la misma escala que la normal.

4.3 Un ejemplo con números

Caso	Etiqueta	Predicción	Sin ponderar	Ponderada
Error en la clase minoritaria	1	0,2	1,609	**2,024**, × 1,258
El mismo error en la mayoritaria	0	0,8	1,609	1,336, × 0,83

El mismo nivel de error cuesta más si ocurre en la clase con menos muestras.

5. Cómo se entrena

- **Descenso de gradiente:** se calcula cuánto cambia la pérdida al mover cada peso y se mueven todos en la dirección que la reduce.
- **Tasa de aprendizaje:** el tamaño de cada paso. Aquí es 0,5, alta porque el problema es pequeño y casi separable.
- **Adam:** adapta el paso de cada peso según la historia de sus gradientes.
- **Lotes y épocas:** `fit` divide las 83 muestras en lotes de 32 y actualiza los pesos después de cada lote: 3 pasos por época. Una época es una pasada por todos los datos.

> **OJO · POR QUÉ LA PRIMERA ÉPOCA REPORTA 0,94 Y NO 0,69**
>
> Con los pesos en cero, la predicción inicial es 0,5 y la pérdida es ln 2 ≈ 0,69. Pero `fit` reporta el **promedio de los lotes** de la época mientras los pesos cambian, y con una tasa de 0,5 los primeros pasos sobrepasan el mínimo. Por eso la época 0 muestra 0,9399663. Un bucle manual con un solo paso por época da 0,6931 y no coincide con la salida esperada.

PARTE III

La solución, celda por celda

El código tal como quedó, explicado línea por línea.

6. Preparación de los datos

CELDA 11 **Carga, filtro, escalado y división**

CÓDIGO

```
data = pd.read_csv('wine.data', sep=",", header=None)
X_all = np.array(data.iloc[:,1:])
y_all = np.array(data.iloc[:,0])
X = X_all[np.where((y_all==2)|(y_all==3))]
y = y_all[np.where((y_all==2)|(y_all==3))]-2
scaler = preprocessing.MinMaxScaler((0, 1))
X = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = model_selection.train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42)
```

Parte del código	Qué hace
`data.iloc[:,1:]` y `data.iloc[:,0]`	Separan las 13 variables de la etiqueta, que está en la primera columna
`np.where(...)` y `- 2`	Filtran los vinos tipo 2 y 3 y los recodifican a 0 y 1
`MinMaxScaler((0, 1))`	Lleva cada variable al rango de 0 a 1
`stratify=y, random_state=42`	Mantiene la proporción de clases en ambos conjuntos y fija la partición

SALIDA DE LAS CELDAS 13 Y 15

```
Número de muestras de entrenamiento = 83
Número de muestras de prueba = 36
Número de características del conjunto de datos = 13
Número de muestras de la clase 2 en entrenamiento = 50
Número de muestras de la clase 3 en entrenamiento = 33
Número de muestras de la clase 2 en prueba = 21
Número de muestras de la clase 3 en prueba = 15
```

OJO · FUGA DE INFORMACIÓN MENOR

El escalador se ajusta con los 119 vinos antes de dividir, así que «conoce» el rango de la prueba. Lo correcto sería ajustarlo solo con el entrenamiento. Venía así en la plantilla y no se cambió.

7. Punto 1 · class_weights

QUÉ SE PEDÍA

Una lista con el peso de cada clase según el desbalance de las etiquetas de entrenamiento.

CÓDIGO · CELDA 20

```
def class_weights(y):
    import numpy as np
    import tensorflow as tf
    import keras
    n_0 = np.sum(y == 0)
    n_1 = np.sum(y == 1)
    w_0 = float((n_0 + n_1) / (2 * n_0))
    w_1 = float((n_0 + n_1) / (2 * n_1))
    weights_list = [w_0, w_1]
    return weights_list
```

Parte del código	Qué hace
`import numpy, tensorflow, keras`	Necesarios porque el calificador ejecuta la función aislada
`np.sum(y == 0)`	Cuenta las muestras de cada clase: compara y suma los verdaderos
`(n_0 + n_1) / (2 * n_k)`	Aplica la fórmula del peso
`float(...)`	Convierte a número de Python, como pide el enunciado

SALIDA

```
[0.83    1.25758]
```

8. Punto 2 · get_loss

QUÉ SE PEDÍA

Una función que devuelva otra función: la entropía cruzada ponderada, que solo reciba `y_true` y `y_pred`.

CÓDIGO · CELDA 27

```
def get_loss(class_weights):
    import keras
    w_0, w_1 = class_weights
    def loss(y_true, y_pred):
        y_true = keras.ops.cast(y_true, y_pred.dtype)
        eps = keras.backend.epsilon()
        y_pred = keras.ops.clip(y_pred, eps, 1 - eps)
        w_bce = -keras.ops.mean(w_1 * y_true * keras.ops.log(y_pred)
                                + w_0 * (1 - y_true) * keras.ops.log(1 - y_pred))
        return w_bce
    return loss
```

Parte del código	Qué hace
`def get_loss(class_weights)`	Recibe solo los pesos. La plantilla traía `get_loss(y_true, y_pred, class_weights)`, que no coincide con las pruebas
`def loss(y_true, y_pred)`	Función interna con la firma que exige Keras para una pérdida
`keras.ops.cast(y_true, y_pred.dtype)`	Iguala el tipo de dato de las etiquetas al de las predicciones
`keras.ops.clip(y_pred, eps, 1 - eps)`	Evita `log(0)`, que daría infinito

`w_1 * y_true * log(y_pred)`	Error de la clase 1, ponderado
`w_0 * (1 - y_true) * log(1 - y_pred)`	Error de la clase 0, ponderado
`-keras.ops.mean(...)`	Promedia sobre las muestras y cambia el signo

INTUICIÓN · EL PATRÓN CLOSURE

Keras llama a la pérdida solo con `(y_true, y_pred)`. Para darle los pesos, se crea una función externa que los recibe y devuelve una interna que los «recuerda». Es un patrón común para pérdidas y métricas con parámetros.

SALIDA DE LA CELDA 29

```
weighted_bce = 0.6931471805599454
```

Es ln 2: con predicciones de 0,5 cada muestra aporta ln 2 y, como el peso promedio es 1, la ponderación no cambia el total.

9. Punto 3 · optimizer

CÓDIGO · CELDA 38

```
def optimizer(type_opt, learning_rate):
    import numpy as np
    import tensorflow as tf
    import keras
    if type_opt == 'SGD':
        opt = keras.optimizers.SGD(learning_rate=learning_rate)
    elif type_opt == 'Adam':
        opt = keras.optimizers.Adam(learning_rate=learning_rate)
    elif type_opt == 'RMSprop':
        opt = keras.optimizers.RMSprop(learning_rate=learning_rate)
    else:
        raise ValueError("type_opt debe ser 'SGD', 'Adam' o 'RMSprop'")
    return opt
```

Optimizador	Cómo actualiza los pesos
`SGD`	Un paso del mismo tamaño en la dirección del gradiente
`Adam`	Paso adaptado por peso, con promedios del gradiente y de su cuadrado
`RMSprop`	Paso normalizado por el promedio reciente del gradiente al cuadrado

El `raise ValueError` evita que un nombre mal escrito pase en silencio.

10. Punto 4 · get_model

CÓDIGO · CELDA 46

```
def get_model(n_inputs, n_neurons, optimizer, loss):
    import numpy as np
    import tensorflow as tf
    import keras
```

```
    model = keras.models.Sequential([
        keras.layers.Input(shape=(n_inputs,)),
        keras.layers.Dense(n_neurons, activation='sigmoid')
    ])
    model.compile(optimizer=optimizer, loss=loss)
    return model
```

Parte del código	Qué hace
`Sequential([...])`	Modelo de capas en secuencia
`Input(shape=(n_inputs,))`	Entrada de 13 variables
`Dense(n_neurons, activation='sigmoid')`	Una neurona con sigmoide: 13 pesos y 1 sesgo
`model.compile(optimizer, loss)`	Asocia el optimizador y la pérdida ponderada

SALIDA DE LA CELDA 47

```
dense_28 (Dense)    (None, 1)    14
Total params: 14 (56.00 B)
```

11. Punto 5 · train_model

CÓDIGO · CELDA 53

```
def train_model(model, epochs, optimizer, X_t, Y_t, loss):
    import numpy as np
    import tensorflow as tf
    import keras
    history = model.fit(X_t, Y_t, epochs=epochs, verbose=0)
    losses = history.history['loss']
    return losses, model.get_weights()
```

Parte del código	Qué hace
`model.fit(X_t, Y_t, epochs=epochs, verbose=0)`	Entrena sin imprimir. El modelo ya trae optimizador y pérdida desde `get_model`
`history.history['loss']`	Lista con la pérdida promedio de cada época
`model.get_weights()`	Devuelve la matriz de pesos y el sesgo

Los argumentos `optimizer` y `loss` no se usan dentro, pero la firma se mantiene porque así la llama el calificador.

SALIDA DE LA CELDA 54

```
Epoch 0 loss = 0.9399663209915161
Epoch 1 loss = 0.3858067989349365
w[0] = [2.2023726]
b = [-0.09913211]
```

Iguales a la salida esperada, salvo el sesgo, que difiere en la séptima cifra por la versión de TensorFlow.

12. Punto 6 · model_predict

CÓDIGO · CELDA 60

```
def model_predict(model, X_test):
    import numpy as np
    import tensorflow as tf
    import keras
    y_pred = model(tf.convert_to_tensor(X_test, dtype=tf.float32))
    return y_pred
```

Parte del código	Qué hace
`def model_predict(model, X_test)`	La plantilla traía `model_predict(log_reg, b, w, X_test)`; se siguió la firma que usan las pruebas
`tf.convert_to_tensor(..., dtype=tf.float32)`	Convierte los datos en un tensor de decimales
`model(...)`	Aplica el modelo y devuelve un tensor de probabilidades

SALIDA DE LA CELDA 62

```
Primeras dos predicciones:
 [[0.9925333]
 [0.0018804]]
Accuracy sobre X_test después de 10 epochs: 0.9879518
```

OJO · ESA ACCURACY ES DE ENTRENAMIENTO

El texto dice «sobre X_test», pero la celda evalúa `X_t`, el entrenamiento: 0,9879518 es exactamente 82 de 83. La accuracy en prueba está en la sección 17.

PARTE IV

La intervención: cómo se llegó al 100 %

Qué falló, cómo se averiguó la causa y qué se cambió.

13. Las cuatro correcciones

Corrección 1 · Firmas que no coinciden con las pruebas

PROBLEMA

La plantilla definía `get_loss(y_true, y_pred, class_weights)` y `model_predict(log_reg, b, w, X_test)`, pero las celdas de prueba las llaman como `get_loss(class_weights)` y `model_predict(model, X_t)`.

SOLUCIÓN

Se leyeron las celdas de prueba y se siguió su forma de llamar a las funciones. Ante una contradicción entre la firma de la plantilla y el uso en las pruebas, mandan las pruebas.

Corrección 2 · Los `import` dentro de cada función

PROBLEMA

Una prueba dio **0** con el mensaje `name 'tf' is not defined`.

ANTES · 0, ESQUEMA SIN IMPORT

```
def optimizer(type_opt, learning_rate):
    if type_opt == 'Adam':
        opt = keras.optimizers.Adam(...)
```

DESPUÉS · 100

```
def optimizer(type_opt, learning_rate):
    import numpy as np
    import tensorflow as tf
    import keras
    if type_opt == 'Adam': ...
```

POR QUÉ FUNCIONA

El calificador ejecuta cada función en su propio espacio y no ve los `import` del notebook. Importar dentro hace a cada función autosuficiente.

Corrección 3 · El tipo de dato de la pérdida

PROBLEMA

Una versión de `get_loss` que convertía todo a `float32` con `tf.cast` obtuvo **10**, con el mensaje «El tipo de dato retornado al llamar loss no es el esperado».

SOLUCIÓN

Se reescribió con `keras.ops` y convirtiendo `y_true` al tipo de `y_pred`, en lugar de forzar un tipo fijo. Así la pérdida devuelve el mismo tipo que las predicciones que recibe; la prueba usa predicciones en `float64`.

Corrección 4 · Entrenar con `fit`, no con un bucle manual

Método	Pérdida en la época 0	¿Coincide con 0,9399663?
Bucle con `GradientTape`, un paso con todos los datos por época	0,6931	No
`model.fit`, lotes de 32 y 3 pasos por época	0,9399663	Sí ✓

La salida esperada corresponde al promedio de los lotes que reporta `fit`, como se explicó en la sección 5.

14. Lecciones

LECCIONES

1. **Las celdas de prueba dicen cómo se llamarán las funciones.** Leerlas antes de escribir evita errores de firma.
2. **El calificador ejecuta cada función aislada**: cada una debe importar lo que usa.
3. **El tipo de dato es parte del resultado.** Es mejor adaptarse al tipo de la entrada que forzar uno fijo.
4. **«Entrenar» admite varias formas**, y solo una reproduce la referencia. Comparar con la salida esperada ayuda a elegir.

PARTE V

Resultados y análisis

Qué aprendió el modelo y qué tanto ayudó la ponderación.

15. Las pruebas

Punto	Función	Pruebas	Nota
1	`class_weights`	1_1, 1_2	100, 100
2	`get_loss`	2_1, 2_2	100, 100
3	`optimizer`	3_1, 3_2	100, 100
4	`get_model`	4_1, 4_2	100, 100
5	`train_model`	5_1, 5_2	100, 100
6	`model_predict`	6_1, 6_2	100, 100
Envío oficial con `submit_task`			**100 %**

16. Curva de pérdida

Pérdida ponderada por época · menor es mejor

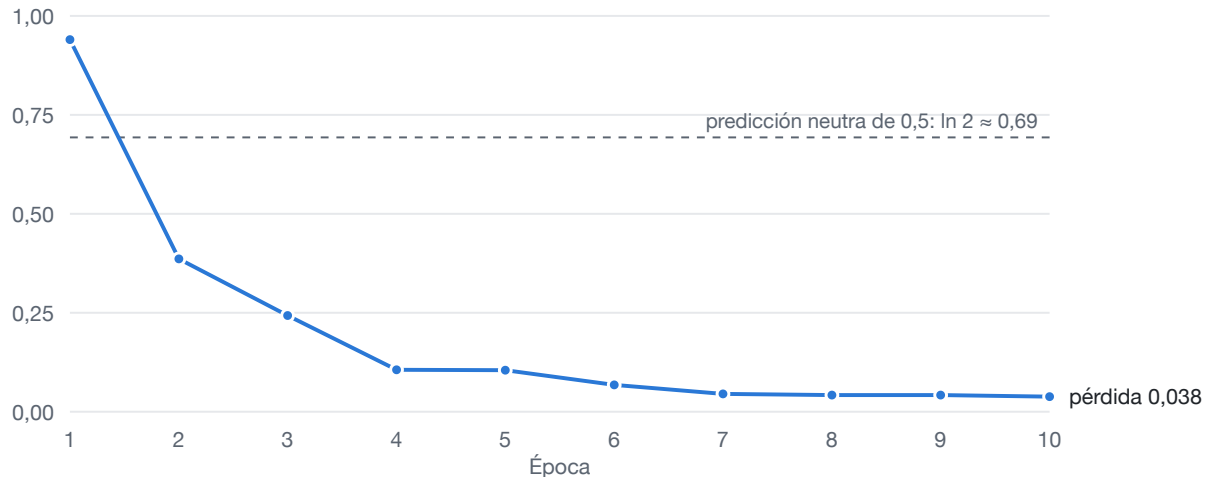


Figura 1. Pérdida de las 10 épocas, en una reproducción con las mismas versiones, semillas y pesos iniciales en cero. Las épocas 1 y 2 coinciden con las que imprime el notebook. La línea de trazos es la pérdida de la predicción neutra.

La mayor caída ocurre en las primeras 4 épocas. Después el modelo ya separa casi todas las muestras y la mejora es marginal, de 0,106 a 0,038.

17. Errores y variables

17.1 Matrices de confusión

	Entrenamiento · predice 0	Entrenamiento · predice 1	Prueba · predice 0	Prueba · predice 1
Real 0, vino tipo 2	49	1	18	3
Real 1, vino tipo 3	0	33	0	15

- **98,8 % en entrenamiento**, 82 de 83, y **91,7 % en prueba**, 33 de 36.
- **Ningún vino de la clase minoritaria se clasificó mal**: su recall es del 100 % en ambos conjuntos.
- Todos los errores van en la misma dirección: vinos tipo 2 clasificados como tipo 3.
- Con 36 muestras de prueba, cada error vale 2,8 puntos: la brecha de 7 puntos son 3 vinos.

17.2 Qué variables usa el modelo

Variable	Peso	Favorece a	Media, tipo 2	Media, tipo 3
Flavanoids	−6,01	Tipo 2	2,08	0,78
OD280/OD315	−5,34	Tipo 2	2,79	1,68
Color intensity	+5,08	Tipo 3	3,09	7,40
Hue	−4,46	Tipo 2	1,06	0,68
Malic acid	+3,88	Tipo 3	1,93	3,33
Proanthocyanins	−3,08	Tipo 2	1,63	1,15

El signo de cada peso coincide con los datos: el vino tipo 3 tiene más del doble de intensidad de color y menos de la mitad de flavonoides. El modelo aprendió diferencias químicas reales.

18. ¿Sirvió ponderar la pérdida?

Para comprobarlo se entrenó el mismo modelo con pesos `[1, 1]`, con la misma semilla y configuración:

Pérdida	Entrenamiento	Prueba	Matriz de prueba
Sin ponderar	98,8 %	91,7 %	`[[18, 3], [0, 15]]`
Ponderada	98,8 %	91,7 %	`[[18, 3], [0, 15]]`

CONCLUSIÓN

Las predicciones son idénticas. Las dos clases se separan casi perfectamente con una frontera lineal, y un desbalance moderado, 60 / 40, no alcanza a cambiar ninguna decisión. La ponderación sigue siendo la técnica correcta en general; su efecto se notaría con un desbalance más fuerte o con clases menos separables. **Para afirmar que una técnica funcionó hay que compararla con no aplicarla.**

PARTE VI

Para consolidar

Preguntas, glosario y conclusiones.

19. Preguntas de autoevaluación

Conviene intentar responder antes de leer la respuesta.

1. ¿Por qué una regresión logística es una red neuronal?

Porque es una sola neurona: suma ponderada de las entradas más un sesgo, seguida de una sigmoide.

2. ¿Por qué el peso de la clase 1 es mayor?

Porque tiene menos muestras, 33 frente a 50. Con `w_k = N / (2 n_k)`, menos muestras implica más peso.

3. ¿Por qué la pérdida con predicciones de 0,5 da ln 2 aunque esté ponderada?

Cada muestra aporta $-\ln 0{,}5 = \ln 2$, y el peso promedio por muestra es exactamente 1.

4. ¿Para qué sirve `clip(y_pred, ε, 1 − ε)`?

Para evitar `log(0)`, que daría infinito y rompería el entrenamiento.

5. ¿Por qué `get_loss` devuelve una función?

Porque Keras llama a la pérdida solo con `(y_true, y_pred)`; la función interna recuerda los pesos.

6. ¿Por qué cada función importa `keras` dentro?

Porque el calificador ejecuta cada función aislada y no ve los `import` del notebook.

7. ¿Por qué la época 0 reporta 0,94 si la pérdida inicial es 0,69?

`fit` reporta el promedio de los 3 lotes de la época mientras los pesos cambian, y con tasa 0,5 los primeros pasos sobrepasan el mínimo.

8. ¿Qué significa un peso de −6,01 en flavonoides?

Que más flavonoides empujan fuertemente hacia el vino tipo 2: el tipo 3 tiene menos de la mitad.

9. ¿La ponderación mejoró el modelo?

En este caso no: sin ponderar se obtienen las mismas predicciones, porque las clases son casi separables.

20. Glosario

Término	Definición breve
Regresión logística	Modelo de una neurona con sigmoide para clasificación binaria
Sigmoide	Función que lleva cualquier número al rango de 0 a 1
Entropía cruzada binaria	Pérdida que castiga con −log la probabilidad asignada a la clase correcta
Pesos por clase	Factores que hacen pesar más los errores de la clase minoritaria
Closure	Función interna que recuerda variables de la función que la creó
Tasa de aprendizaje	Tamaño de cada paso del optimizador
Época	Una pasada completa por los datos de entrenamiento
Lote	Grupo de muestras tras el cual se actualizan los pesos; 32 por defecto
Recall	Proporción de muestras de una clase que el modelo reconoce
Fuga de información	Usar datos de prueba, aunque sea indirectamente, durante la preparación o el entrenamiento

21. Conclusiones

- **El problema:** clasificar dos tipos de vino con clases desbalanceadas.
- **La solución:** una regresión logística en Keras con entropía cruzada ponderada, entrenada con Adam.
- **La intervención:** corregir dos firmas de la plantilla, importar las librerías dentro de cada función, respetar el tipo de dato en la pérdida y entrenar con `fit` para reproducir la referencia.
- **El resultado:** 12 de 12 pruebas y 100 %. El modelo acierta el 98,8 % en entrenamiento y el 91,7 % en prueba, sin errores en la clase minoritaria.
- **Lo que enseña el análisis:** en este caso la ponderación no cambió ninguna predicción. Es la técnica correcta, pero su efecto depende de qué tan separables y desbalanceadas estén las clases.

Créditos del notebook original: profesor Fabio Augusto González, asistentes docentes Juan Sebastián Malagón Torres y Santiago Toledo Cortés, diseño de imágenes de Mario Andrés Rodríguez Triana y coordinador de virtualización Edder Hernández Forero. Universidad Nacional de Colombia, Facultad de Ingeniería.